

Course Name: Computer Graphics Fundamentals

Project Title: Synthwave Drive

Team Members: Almas Alkhan, SE-2425
Turar Nurbauli
Muslim Syzdykov
Nursalim Onalbayev

Instructor: Nazira Shaimerdenova

Progress report: 7 November 2026
Final submission date to be confirmed

Contents

1 Introduction and project description — 3

2 Literature review and background — 4

3 Technical implementation — 5

4 Graphics and rendering — 6

5 Rendering results — 7

6 Testing and evaluation — 8

7 Conclusion and future work — 9

8 References — 10

This progress report documents the implemented framework and separates tested results from team work still required for the final defense. It is not a claim that the final project already satisfies every requirement.

The instructor’s template supplies the cover, typography and section topics. The body includes actual source excerpts, framebuffer captures, local measurements and a comparative literature review. Source code and evidence correspond to the local update on 7 November 2026.

1 Introduction and project description

Synthwave Drive is a four-member Computer Graphics Fundamentals final project based on Topic 13. Its intended scene combines a scrolling neon road, a steerable low-poly car, mountains, a gradient sky and a striped sun. The current implementation provides the common rendering framework and Almas Alkhan's road and camera contribution. Car geometry, hierarchy, environment and lighting are not yet integrated in this checkout.

1.1 Objective

Build a reproducible C++ application using modern OpenGL, shared model/view/projection transforms, procedural graphics and interactive viewing. The framework should allow the remaining members to add their modules without duplicating the main loop, window, input handler or shader loader.

1.2 Problem statement

An apparently endless road must move smoothly without allocating geometry every frame. Camera input must respond consistently across frame rates and window sizes. All future modules must share coordinate conventions and resource ownership so that lighting, scene transforms and depth testing remain compatible during integration.

1.3 Scope and contributions

Implemented: an indexed road mesh, a stationary reference block, procedural grid shading, follow and orbit views, keyboard/mouse input, speed control, pause, palette selection, reset, live FPS/help, portable shader lookup and automated local checks. The block only communicates scale and depth; it is not a car.

Almas owns the framework, road, cameras and integration. Turar Nurbauli is assigned the car, steering and parent–child transforms. Muslim Syzdykov is assigned mountains, sky and sun. Nursalim Onalbayev is assigned two-light shading, effects and validation. These assignments do not imply that the pending modules have already been delivered.

R1, R5 and R6 have local implementation evidence. R2 needs at least five distinct scene objects; R3 still needs a visible hierarchy; R4 still needs two lights and per-fragment shading. R7 and R8 require validation on the TA setup after integration. No stretch effects are claimed.

2 Literature review and background

Gouraud [1] interpolates vertex intensities, whereas Phong [2] evaluates a local illumination model using surface orientation. Per-fragment evaluation is appropriate for the required moving highlights because a highlight can fall between vertices. The planned car shader will use Phong or Blinn–Phong; the current emissive grid does not implement either lighting model.

Cook and Torrance [3] represent material reflection through microfacet behavior, while Whitted [4] expands illumination with recursive reflection and refraction. Schlick [5] reduces the cost of a physically motivated reflectance approximation. Compared with these approaches, the required local lighting model is easier to inspect and control for a stylized scene, but does not provide global light transport or a physically measured car finish.

Guarnera et al. [6] distinguish analytic and acquired reflectance representations. This supports using explicit material presets for a small educational scene rather than claiming measured paint. Han et al. [7] address nonlinear filtering of normal-map shading, while Tan et al. [8] address resolution-dependent reflectance. Our derivative-filtered grid solves a narrower problem: keeping thin procedural lines readable under minification. It does not implement either paper's reflectance filtering.

Jakob et al. [9] model layered materials, and Heitz et al. [10] account for multiple microfacet scattering. These methods explain effects that an ambient term cannot reproduce. Heitz et al. [11] instead target real-time polygonal area-light shading with linearly transformed cosines. The team's planned two-point-light solution is simpler and matches R4, while lacking extended-light integration and layered transport.

Belcour and Barla [12] model varying iridescence; Jakob et al. [13] model discrete stochastic microfacets; Yan et al. [14] model detailed specular glints. These target angular and microscopic variation beyond a single shininess parameter. Belcour [15] uses statistical operators for efficient layered materials. Such methods could improve a later car finish but introduce complexity beyond the present framework milestone.

The comparison therefore favors inexpensive procedural surfaces and explicit local lighting for this project. The bibliography contains 15 journal articles, but journal quartiles for an agreed ranking year/category have not been verified. The final submission must document Q1–Q2 eligibility and replace any ineligible sources before claiming the literature requirement is complete.

3 Technical implementation

3.1 Tools and technologies

C++17 manages application state and GPU-resource lifetimes. GLFW creates the window and receives events; GLEW loads OpenGL entry points. CMake builds the executable and copies external GLSL files beside it. macOS requests a 4.1 core context while shaders use GLSL 330-compatible features. No new runtime dependency was added to the earlier GLFW/GLEW assignment setup.

3.2 Architecture and development process

Window owns the context. Mesh owns VAO/VBO/EBO resources. Shader compiles and links programs with filename-specific errors. Input converts callbacks into held axes and one-shot actions; SceneState stores common state. Camera builds view/projection matrices; Road and ReferenceMarker render scene geometry; Hud renders FPS and controls last. GPU owners are declared after Window so they are destroyed before context teardown.

The update replaces duplicated road vertices with indexed geometry, adds interactive orbit controls, and introduces performance instrumentation. Tests exercise CPU state transitions and actual OpenGL framebuffer output. The rendering remains intentionally modular so teammate work can be integrated without claiming their unfinished features.

3.3 Indexed geometry and transformations

```
glBindVertexArray(vao_);
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, ebo_);
glBufferData(GL_ELEMENT_ARRAY_BUFFER, indices.size() *
             sizeof(std::uint32_t), indices.data(), GL_STATIC_DRAW);
glDrawElements(GL_TRIANGLES, count_, GL_UNSIGNED_INT, nullptr);
```

The road stores four vertices and six indices for two triangles. Static data is uploaded once. The VAO retains the element-buffer binding; drawing reuses it. Mesh initialization rejects invalid triangle layouts and out-of-range indices.

```
clipPosition = projection * view * model * position;
childWorld = parentWorld * childLocal;
```

The first equation describes the implemented column-vector convention; matrices are stored column-major and uploaded without transposition. The second is the required future car hierarchy, not an implemented result. Normal transformation uses the inverse transpose of the model’s 3×3 linear part; the math helper is tested for nonuniform scaling.

4 Graphics and rendering

4.1 Procedural surface and animation

The road shader receives UV coordinates derived from ground position. Its repeating grid advances through a bounded phase rather than by moving or rebuilding the mesh. A two-unit grid period and a wrapped offset keep coordinates numerically stable. Positive speed moves the visible pattern toward +Z, matching the intended forward drive illusion.

```
phase = fmod(phase + speed * dt / gridWorldSize, 1.0);
patternCoordinate = uv - vec2(0.0, phase);
```

The fragment shader uses periodic line masks and fwidth-based filtering near pixel boundaries. Cyan lines mark the driving region; magenta lines and highlighted edges distinguish the surroundings. G changes the palette. A distance fade reduces high-frequency detail near the horizon. This is a procedural shader surface, not an image texture or bloom effect.

4.2 Camera and input

```
eye = target + radius * vec3(cos(pitch) * sin(yaw),
                             sin(pitch),
                             cos(pitch) * cos(yaw));
view = lookAt(eye, target, worldUp);
```

The orbit camera uses yaw, pitch and radius. Pitch is limited to 0.08–1.4 radians and radius to 3–45 units. Held keyboard input is integrated with delta time; mouse displacement is consumed once without another delta-time multiplication. Mouse and scroll input select orbit mode automatically. Framebuffer dimensions determine the perspective aspect ratio, and zero-sized buffers skip rendering.

W/S controls speed; Space freezes road phase while the camera remains usable. R resets state and phase. H hides help but preserves the live FPS display. Simulation steps are clamped to 0.05 seconds after stalls. The HUD uses original compact glyph arrays expanded into triangles and restores depth state after drawing.

4.3 Lighting and shading status

The current grid and reference block use unlit colors. R4 remains pending: Nursalim must implement per-fragment ambient, diffuse and specular contributions from at least two lights, with one movable or toggleable. Camera and lights must use the same world-space convention. Emission alone does not satisfy this requirement.

5 Rendering results

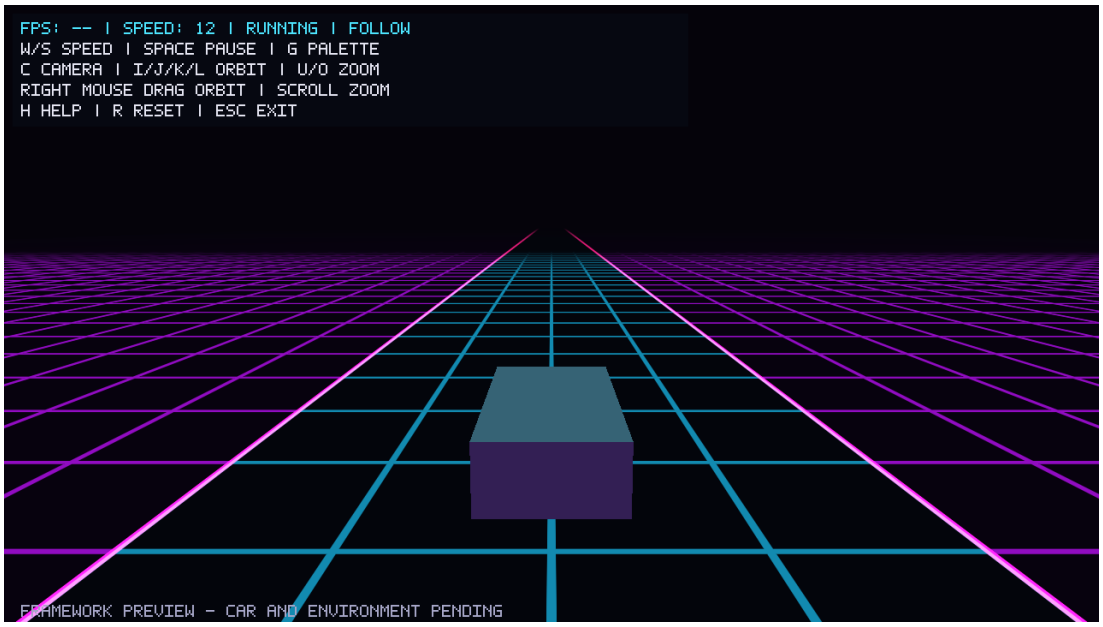


Figure 1. Actual follow-view framebuffer with grid, reference block and controls.

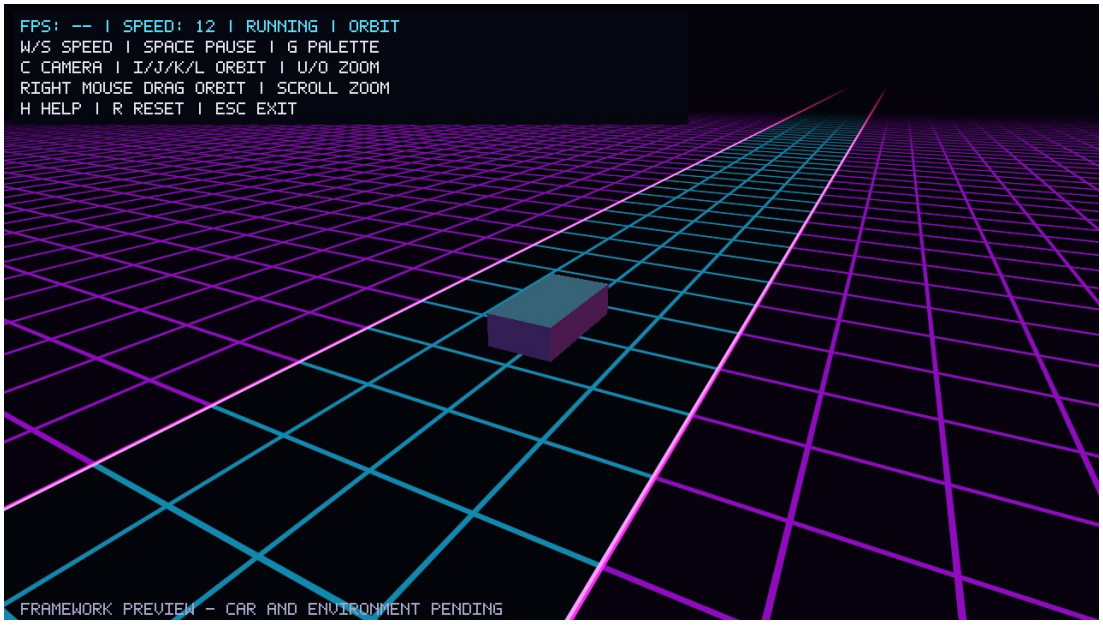


Figure 2. Orbit presentation view of the same scene and indexed road.

These deterministic captures show the current geometry and interface. FPS is intentionally shown as “--” because a single-frame capture does not measure throughput. Interactive execution displays a numerical FPS value. The final demo must retain that live value throughout recording.

6 Testing and evaluation

6.1 Testing methodology

The CPU executable passes 40 checks covering matrix arithmetic, normal transforms, view/projection, grid phase, frame-time bounds, input edges, reset, orbit bounds and mouse-event consumption. OpenGL smoke checks render actual frames to verify visible output, phase movement/repetition, camera changes, speed, reset, portrait resizing, mouse orbit, pause, palette and GL errors at checked stages.

```
ctest --test-dir build --output-on-failure
./build/synthwave_drive --smoke-test build/smoke
./build/synthwave_drive --benchmark 2400
```

6.2 Performance evaluation

The local Release benchmark uses Apple M3 Pro, Apple Clang 17 and OpenGL 4.1 Metal. It discards 60 warm-up frames and samples 2400 frames. Vsync is disabled; glFinish waits for GPU completion. Timing includes event polling, update, rendering, buffer swap and completion, but excludes the occasional window-title update after the sample. Only the lightweight framework is measured.

Measurement	Local prototype result
Framebuffer	1280 × 720
Mean throughput	1388.04 frames/s
95th percentile frame time	1.45321 ms
Worst sampled frame time	12.5889 ms
CPU suite	40 checks passed
TA hardware and integrated scene	Not yet measured

This short uncapped benchmark is not a sustained TA-machine test or a final-scene performance result. R7 still needs stable ≥30 FPS on the TA machine at 1280×720 after integration; ≥60 FPS is the full-performance target. Record effect settings and longer-run results before submission.

6.3 Issues and debugging

Initial macOS hidden-window captures used a stale Retina framebuffer size and showed a cropped scene. Disabling Retina framebuffer scaling and processing initial window events before readback corrected the capture to 1280×720. Portrait and zero-size checks guard projection resizing. Shader errors identify files; unavailable shaders fail clearly instead of silently drawing a blank scene.

7 Conclusion and future work

The current framework gives the team a tested starting point with indexed rendering, procedural motion, keyboard/mouse viewing and visible performance information. It is suitable for module integration, but does not yet satisfy the complete final-project specification.

7.1 Remaining implementation

Turar must add a steerable car with parent–child transformations and wheel motion. Muslim must add distinct mountain formations and a sky/sun composition. Nursalim must add two-light per-fragment shading and one movable or toggleable light. Almas must integrate the modules and verify the final object inventory, controls, rendering order, build and performance. Bloom and CRT remain optional stretch targets until implemented and tested.

7.2 Final submission preparation

Submit three days before the confirmed defense date by 23:59 Asia/Almaty. The exact date is not yet supplied. A human must verify TA repository access and create the v1.0 tag on the submission date. All four members must make their own reviewed contributions and be able to explain shared code.

Replace prototype captures and measurements with final-scene evidence. Verify Q1–Q2 journal eligibility, expand actual contribution records, prepare 8–12 slides for a 7–10 minute presentation, and record a 3–6 minute 1080p real-time demo with live FPS throughout and feature narration or captions. Slides and demo are not completed by this report.

7.3 AI Use

OpenAI Codex assisted with Almas’s framework implementation, indexed mesh conversion, orbit controls, HUD, tests, local build checks, documentation and this report draft. Human revisions for this update have not yet been recorded; no completed human review is implied. Before submission, members must review and explain the code and record specific changes they made. The assistant did not stage, commit, push, tag, open pull requests or publish anything. Earlier repository publication was performed manually by Almas.

8 References

[1] H. Gouraud. Continuous Shading of Curved Surfaces. IEEE Transactions on Computers, 20(6), 623-629, 1971. https://doi.org/10.1109/T-C.1971.223313

[2] B. T. Phong. Illumination for Computer Generated Pictures. Communications of the ACM, 18(6), 311-317, 1975. https://doi.org/10.1145/360825.360839

[3] R. L. Cook and K. E. Torrance. A Reflectance Model for Computer Graphics. ACM Transactions on Graphics, 1(1), 7-24, 1982. https://doi.org/10.1145/357290.357293

[4] T. Whitted. An Improved Illumination Model for Shaded Display. Communications of the ACM, 23(6), 343-349, 1980. https://doi.org/10.1145/358876.358882

[5] C. Schlick. An Inexpensive BRDF Model for Physically-based Rendering. Computer Graphics Forum, 13(3), 233-246, 1994. https://doi.org/10.1111/1467-8659.1330233

[6] D. Guarnera, G. C. Guarnera, A. Ghosh, C. Denk and M. Glencross. BRDF Representation and Acquisition. Computer Graphics Forum, 35(2), 625-650, 2016. https://doi.org/10.1111/cgf.12867

[7] C. Han, B. Sun, R. Ramamoorthi and E. Grinspun. Frequency Domain Normal Map Filtering. ACM Transactions on Graphics, 26(3), Article 28, 2007. https://www.cs.columbia.edu/cg/normalmap/

[8] P. Tan, S. Lin, L. Quan, B. Guo and H. Y. Shum. Filtering and Rendering of Resolution-Dependent Reflectance Models. IEEE Transactions on Visualization and Computer Graphics, 14(2), 412-425, 2008. https://www.microsoft.com/en-us/research/publication/filtering-rendering-resolution-dependent-reflectance-models/

8 References continued

[9] W. Jakob, E. d'Eon, O. Jakob and S. Marschner. A Comprehensive Framework for Rendering Layered Materials. ACM Transactions on Graphics, 33(4), Article 118, 2014. https://rgl.epfl.ch/publications/Jakob2014Comprehensive

[10] E. Heitz, J. Hanika, E. d'Eon and C. Dachsbacher. Multiple-Scattering Microfacet BSDFs with the Smith Model. ACM Transactions on Graphics, 35(4), Article 58, 2016. https://doi.org/10.1145/2897824.2925943

[11] E. Heitz, J. Dupuy, S. Hill and D. Neubelt. Real-Time Polygonal-Light Shading with Linearly Transformed Cosines. ACM Transactions on Graphics, 35(4), Article 41, 2016. https://eheitzresearch.wordpress.com/415-2/

[12] L. Belcour and P. Barla. A Practical Extension to Microfacet Theory for the Modeling of Varying Iridescence. ACM Transactions on Graphics, 36(4), Article 65, 2017. https://doi.org/10.1145/3072959.3073620

[13] W. Jakob, M. Hasan, L. Q. Yan, J. Lawrence, R. Ramamoorthi and S. Marschner. Discrete Stochastic Microfacet Models. ACM Transactions on Graphics, 33(4), Article 115, 2014. https://cseweb.ucsd.edu/~ravir/stochastic.pdf

[14] L. Q. Yan, M. Hasan, W. Jakob, J. Lawrence, S. Marschner and R. Ramamoorthi. Rendering Glints on High-Resolution Normal-Mapped Specular Surfaces. ACM Transactions on Graphics, 33(4), Article 116, 2014. https://doi.org/10.1145/2601097.2601155

[15] L. Belcour. Efficient Rendering of Layered Materials using an Atomic Decomposition with Statistical Operators. ACM Transactions on Graphics, 37(4), Article 73, 2018. https://doi.org/10.1145/3197517.3201289